UNIVERSIDAD TECNOLÓGICA DEL PERÚ

CURSO INTEGRADOR I: SISTEMAS SOFTWARE

MULTIPLATAFORMA DE RUTAS DE TRANSPORTE URBANO



INTRODUCCIÓN

Problema

- Transporte urbano informal en Piura (combis/colectivos), sin GPS ni info centralizada.
  Alta espera en hora punta (7–9am / 5:30–7:30pm), rutas a UNP, UDEP, Centro

Solución

App con GPS colaborativo: el pasajero reporta su ubicación al viajar

- Calcula ETA, mapa interactivo (Flutter + flutter_map, polling 5–10s) y dashboard de horas de mayor congestión
  Arquitectura MVC + DAO, principios SOLID

Alcance

- Dentro: login (JWT), rutas/paraderos, ETA, mapa, dashboard, geofencing (opcional)
  Fuera: WebSockets, pagos, verificación de identidad, gestión de empresas



LEAN CANVAS

2. Problema

1. Falta de información confiable sobre rutas
Los usuarios de transporte urbano en Piura no cuentan con datos claros sobre las rutas establecidas de buses y combis.
2. Ausencia de ubicación en tiempo real y tiempo estimado de llegada
No existe forma de saber dónde está el vehículo ni cuánto falta para que llegue, generando incertidumbre en los desplazamientos diarios.
3. Desorden del transporte por falta de paradas obligatorias
La ausencia de paradas establecidas incrementa el caos vial, lo que agrava la congestión y provoca pérdida de tiempo para los usuarios.

1. Segmentos de Clientes:

Estudiantes del secundario universitarios y trabajadores entre 15-50 años de la ciudad de Piura que usan transporte publico, combis o colectivos como medio de transporte habitual hacia el centro de la ciudad, universidades y zonas comerciales.

3. Propuesta de Valor Único:

Visibilidad en tiempo real de las combis y colectivos de Piura impulsada por la misma comunidad. Descubre el tiempo aproximado de llegada de tu transporte y el nivel de tráfico gracias a la ubicación real compartida por otros pasajeros de forma colaborativa.

4. Solución:

Aplicación multiplataforma colaborativa que utiliza el GPS de los propios pasajeros para mapear la trayectoria, paradas habilitadas y posición de las unidades en tiempo aproximado, calculando tiempos estimados de llegada y detectando puntos y horarios críticos de tráfico en Piura, contribuyendo así a una mejor gestión del transporte urbano.

6. Fuentes de Ingresos:

Proyecto académico sin fines de lucro en esta fase; como hipótesis de escalamiento futuro se plantea un modelo freemium o de publicidad local de comercios cercanos a los paraderos.

9. Ventaja competitiva

Modelo de rastreo colaborativo completamente independiente de las empresas de transporte informal, impulsado por una masa inicial de usuarios activos.
Modelo accesible para el usuario final.

7. Estructura de Costos:

Hosting del backend y base de datos (tier gratuito: Render/Railway/Neon), tiempo de desarrollo del equipo, sin costos de licencias por usar herramientas open source (Flutter, flutter_map, FastAPI/Spring Boot).

8. Métricas Clave:

Número de consultas de ETA realizadas, % de usuarios que comparten su ubicación en tiempo real, cobertura de rutas con al menos un colaborador activo, margen de error promedio del ETA calculado, tiempo promedio de espera reportado por paradero, tramos o zonas con mayor congestión detectada, franjas horarias de mayor tráfico por ruta, rutas más consultadas.

5. Canales:

Distribución vía enlace web/APK durante la fase piloto, difusión en redes sociales universitarias y grupos de estudiantes de la zona.

PROJECT CHARTER

Nombre del Proyecto

Multiplaforma de rutas de transporte urbano - Piura

Objetivos del proyecto

Construir una solucion informatica multiplataforma que constribuya a la gestion del trafico y el transporte urbano en Piura, permitiendo consultar rutas, calcular ETA y visualizar horas y tramos de mayor congestion a partir del GPS colaborativo de los propios pasajeros, siguiendo la arquetectura MVC + DAO + SOLID + TDD exigida por el sílabo

Justificación

No existe en Piura una herramienta digital centralizada para el transporte urbano informal; el proyecto responde a una necesidad real de la comunidad universitaria y permite al equipo aplicar de forma integrada las competencias del curso (planificación, diseño, desarrollo, pruebas, despliegue y mantenimiento).



Stakeholders

Equipo de desarrollo (estudiantes), docente del curso (evaluador), usuarios finales (estudiantes y trabajadores que usan transporte urbano, tanto colaboradores que comparten su GPS como consumidores de la información), potencialmente empresas de combis en fases futuras.



Riesgos iniciales

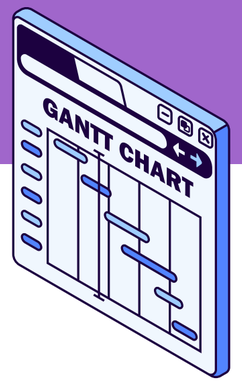
- Retrasos por sobrecarga académica del ciclo.
- Baja adopción inicial de colaboradores activos, lo que reduce la cobertura y precisión del ETA en el piloto, mitigado enfocando el lanzamiento en una comunidad universitaria concentrada.
- Curva de aprendizaje del stack elegido (Flutter/FastAPI o Spring Boot).
- Dependencia de servicios gratuitos (Render/Railway) que pueden tener límites de uso.
- Reportes de ubicación falsos o inconsistentes al no validarse que el colaborador esté físicamente a bordo de la unidad.



Restricciones declaradas

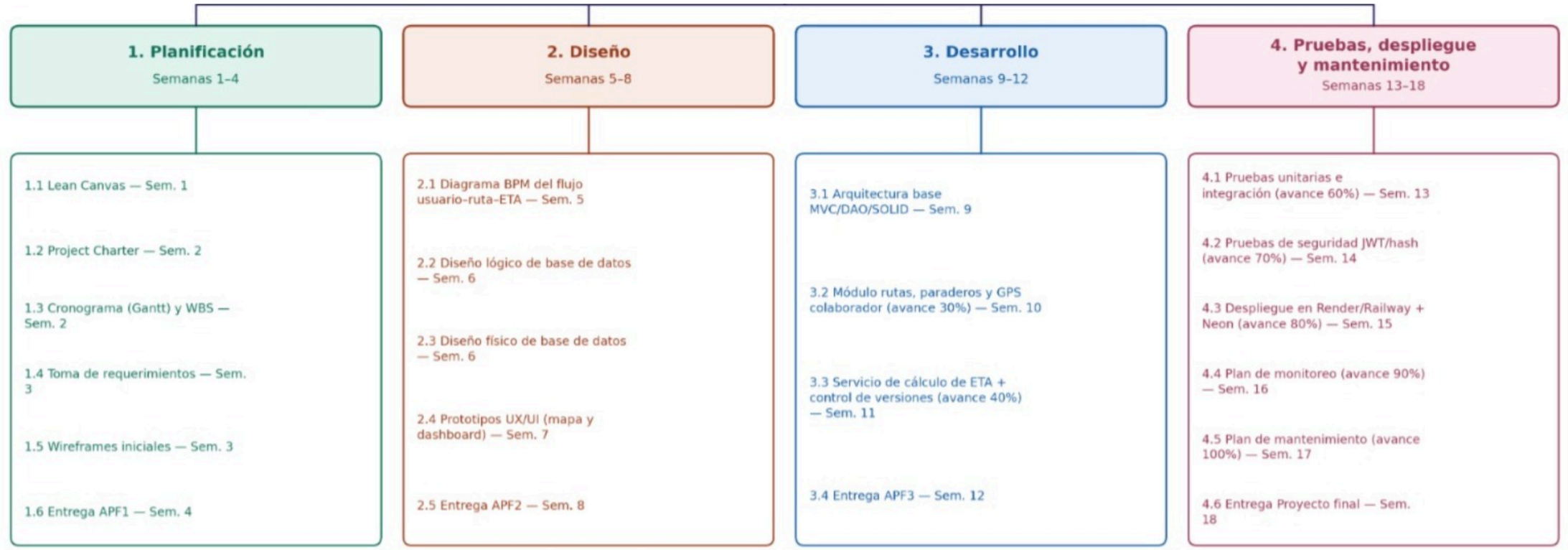
Sin GPS instalado en las combis (se usa el GPS de los pasajeros colaboradores) y sin WebSockets/tiempo real verdadero (se usa polling HTTP)..

DIAGRAMA DE GANTT



Fase / Entregable	Semanas	S1	S2	S3	S4	S5	S6	S7	S8	S9	S10	S11	S12	S13	S14	S15	S16	S17	S18
Unidad I: Análisis y Requerimientos	1 - 4																		
- Lean Canvas y Modelado	1																		
- Project Charter, WBS y Gantt	2																		
- Requerimientos y Prototipos UI	3																		
- Entrega APF1	4																		
Unidad II: Diseño y Arquitectura Base	5 - 8																		
- Diagramas BPM	5																		
- Diseño Lógico y Físico de Base de D	6																		
- Prototipos UX/UI y Documentación	7																		
- Entrega APF2	8																		
Unidad III: Desarrollo y Construcción	9 - 12																		
- Arquitectura Backend (MVC, DAO)	9																		
- Módulo de Rutas y GPS (30%)	10																		
- Control de Versiones e Integración E	11																		
- Entrega APF3	12																		
Unidad IV: Pruebas, Despliegue y Cie	13 - 18																		
- Pruebas Unitarias y Dashboard (60%	13																		
- Pruebas de Seguridad JWT (70%)	14																		
- Despliegue en Cloud (Render/Neon)	15																		
- Monitoreo y Logs del Sistema (90%)	16																		
- Mantenimiento, Backups y Cierre (10	17																		
- Sustentación y Proyecto Final	18																		

Work Breakdown Structure



Toma de Requerimientos

Requerimientos Funcionales



- **RF-01 |** El sistema debe permitir el registro y autenticación de usuarios mediante correo electrónico y contraseña.



- **RF-02** | El sistema debe permitir consultar el listado de rutas de transporte urbano disponibles en Piura.



- **RF-03** | El sistema debe mostrar los paraderos asociados a una ruta seleccionada, en el orden secuencial correcto.



- **RF-04** | El sistema debe permitir que el dispositivo del pasajero colaborador envíe periódicamente su ubicación GPS mientras viaja a bordo de una unidad.

Requerimientos Funcionales



- **RF-05** | El sistema debe calcular y mostrar el tiempo estimado de llegada de una unidad hacia un paradero específico, en base a los reportes GPS más recientes de los colaboradores.



- **RF-06** | El sistema debe visualizar en un mapa interactivo la posición estimada de las unidades activas a partir del GPS compartido por colaboradores (polling cada 5–10 s).



- **RF-07** | El sistema debe presentar un dashboard de "horas calientes" con las franjas horarias y tramos de mayor congestión por ruta, calculado sobre el histórico de reportes GPS.



- **RF-08** | El sistema debe mostrar un indicador de cobertura (número de colaboradores activos) por ruta consultada.



- **RF-09** | El sistema debe permitir enviar una notificación tipo geofencing cuando la unidad seleccionada esté próxima al paradero del usuario.

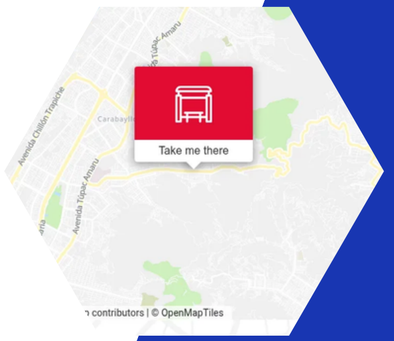
Requerimientos no funcionales



RNF-01 | La actualización de posición de las unidades en el mapa no debe demorar más de 10 segundos respecto al ciclo de polling configurado.

RNF-02 | El sistema debe seguir la arquitectura MVC + DAO aplicando principios SOLID, según lo exigido por el sílabo del curso.

RNF-03 | Las contraseñas de usuario deben almacenarse cifradas (hash) y nunca en texto plano.



RNF-04 | El backend debe exponer documentación de API generada automáticamente (OpenAPI/Swagger).

R**NF-05** | El sistema debe ser desplegable en un proveedor de hosting gratuito (Render/Railway) compatible con el stack elegido.

RNF-06 | El código debe contar con pruebas automatizadas (unitarias y de integración) como parte del proceso de TDD exigido por el curso.



RNF-07 | La interfaz debe ser usable en dispositivos móviles de gama media.

RNF-08 | El sistema debe minimizar el tiempo de retención de la ubicación exacta de cada colaborador y evitar exponer su identidad a otros usuarios.

Clasificación DINO / MoSCoW

Debe (Must, obligatorio)



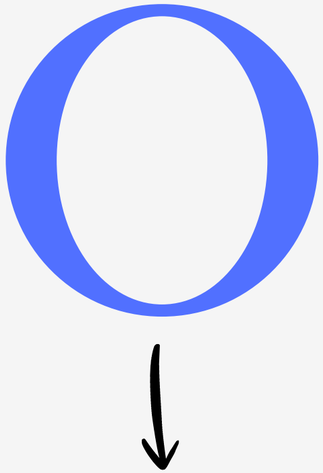
RF-01, RF-02, RF-03, RF-04, RF-05, RF-06
RNF-02, RNF-03, RNF-06

Importante (Should)



RF-07, RF-08
RNF-01, RNF-04, RNF-05, RNF-07, RNF-08

Opcional / Deseable (Could)



RF-09 (notificación geofencing)

No van (Won't)



N-01: Posicionamiento GPS real de combis (hardware instalado por la empresa de transporte)
N-02: Comunicación en tiempo real verdadero vía WebSockets
N-03: Verificación de identidad o validación de que el colaborador esté físicamente a bordo de la unidad
N-04: Integración con sistemas de pago o venta de pasajes
N-05: Gestión administrativa para las empresas de transporte (altas/bajas de unidades en producción real)
Protot

¡GRACIAS!

